



PROJECT REPORT

TERRAGAUGE: ENVIRONMENTAL ACTIVITY LEDGER AND AUDIT PLATFORM

PROGRAMMING IN JAVA

Course Code: CSE2006

Project Category: VITyarthi "Build Your Own Project"

Academic Term: Fall Semester 2026

SUBMITTED BY

Student Name: Soumallaya Mukherjee

Registration Number: 25BAI10226

Program: B.Tech Computer Science and Engineering(AI&ML)

School / Department: SCAI

Institution: VIT Bhopal University, Madhya Pradesh, India

EVALUATED BY

Faculty Mentor: Dr.Adarsh Patel

Course Code: CSE2006

Slot: C11+C12

TABLE OF CONTENTS

1. Executive Summary.....	2
2. Introduction.....	3
3. Problem Statement.....	4
4. Functional Requirements.....	5
4.1 Activity Lifecycle and Ledger Management.....	5
4.2 Calculation and Emission-Factor Resolution.....	5
4.3 Analytical Reporting.....	6
5. Non-functional Requirements.....	7
5.1 Usability.....	7
5.2 Performance.....	7
5.3 Reliability and Data Integrity.....	7
5.4 Maintainability.....	7
6. System Architecture.....	8
7. Design Diagrams.....	9
7.1 Use Case Diagram.....	9
7.2 Workflow Diagram.....	9
7.3 Sequence Diagram.....	10
7.4 Class / Component Diagram.....	10
7.5 ER Diagram.....	11
8. Design Decisions & Rationale.....	12
9. Implementation Details.....	13
10. Screenshots / Results.....	15
10.1 Activity Logging and Ledger.....	15
10.2 Concurrent Batch Calculation.....	15
10.3 Analytics and Reporting.....	16
10.4 Test Execution.....	16
11. Testing Approach.....	17
12. Challenges Faced.....	18
13. Learnings & Key Takeaways.....	19
14. Future Enhancements.....	20
15. References.....	21

2. Introduction

TerraGauge is a Java-based environmental activity ledger and audit platform designed to record resource-consumption activities and calculate their associated greenhouse-gas impact. The system focuses on four activity categories: transportation, electricity, waste, and water.

The application uses Core Java for the application layer and MySQL for persistent storage. JDBC is used to connect the application to the database. Emission factors are maintained in a local CSV dataset so that the reference values can be updated without changing the Java source code.

The system separates activity recording from impact calculation. Activities can be stored first and calculated later as a batch. For batch processing, Java concurrency utilities such as `ExecutorService`, `Callable`, and `Future` are used to process multiple activities concurrently. The resulting impact records are then stored in MySQL and used for reporting.

The project is implemented as a command-line application. The interface provides menu-based activity management, validation messages, calculation controls, and analytical reports. The overall design emphasizes clear separation between presentation, business logic, and data persistence.

3. Problem Statement

Environmental activity data is often collected from different operational sources, while the calculation of corresponding greenhouse-gas impact requires consistent conversion factors and repeatable calculations. A small organisation, laboratory, or academic project may require a local system that can maintain these records without depending on a cloud service.

The project therefore addresses the following requirements:

- **Accessible data management:** Provide a local system for creating, viewing, updating, and deleting environmental activity records.
- **Transparent calculations:** Keep emission factors in a readable dataset and use a clearly defined quantity $\times$ factor calculation.
- **Data integrity:** Validate activity values and maintain relationships between users, activities, and calculated impact records.
- **Batch processing:** Process pending calculations using Java's concurrency facilities rather than relying only on sequential execution.
- **Reporting:** Provide total impact, category-level summaries, and ranked emission sources from the stored data.

4. Functional Requirements

The functional requirements are grouped into three modules.

4.1 Activity Lifecycle and Ledger Management

- **FR 1.1:** The system shall provide a command-line menu for recording activities with category, activity name, quantity, and event date.
- **FR 1.2:** The system shall display stored activities in a structured ledger with identifiers and relevant units.
- **FR 1.3:** The system shall allow an existing activity quantity to be modified and shall invalidate its previous impact calculation.

- **FR 1.4:** The system shall allow an activity to be deleted and shall remove its associated impact record through the database relationship.

4.2 Calculation and Emission-Factor Resolution

- **FR 2.1:** The system shall load reference emission factors from the local CSV dataset.
- **FR 2.2:** The system shall calculate estimated impact using the formula: **Estimated Impact (kg CO₂e) = Activity Quantity × Emission Factor.**
- **FR 2.3:** The system shall identify activities that do not have a current impact calculation and submit them to an ExecutorService thread pool.
- **FR 2.4:** The system shall collect calculation results through Future<ImpactRecord> objects and persist the completed results in a database transaction.

4.3 Analytical Reporting

- **FR 3.1:** The system shall calculate the cumulative carbon footprint stored in the ledger.
- **FR 3.2:** The system shall provide category-wise activity and emission summaries.
- **FR 3.3:** The system shall rank emission sources according to calculated carbon output.

5. Non-functional Requirements

5.1 Usability

The command-line interface uses structured menus, formatted output, contextual prompts, and input validation. Invalid entries are reported to the user without exposing unnecessary exception details.

5.2 Performance

Batch impact calculations are handled separately from activity entry. A fixed thread pool distributes calculation tasks among worker threads, allowing independent activities to be processed concurrently.

5.3 Reliability and Data Integrity

The database schema uses validation constraints and foreign-key relationships. Batch writes use transactions so that a calculation batch can be committed only after the required operations have completed successfully.

5.4 Maintainability

The application separates domain models, database operations, calculation logic, reporting, and presentation. This reduces dependencies between components and allows individual parts of the application to be modified without changing the entire system.

6. System Architecture

TerraGauge follows a three-tier structure consisting of the presentation tier, business-logic and concurrency tier, and data and persistence tier.

Presentation Tier

Main.java handles the command-line interface, menu navigation, user input, validation at the interaction boundary, and display of results.

Business Logic and Concurrency Tier

`ActivityManager.java` manages activity operations, `ImpactCalculator.java` handles emission-factor resolution and concurrent calculations, and `ReportManager.java` performs analytical queries. Domain classes represent users, activities, categories, emission factors, and impact records.

Data and Persistence Tier

`DatabaseManager.java` provides JDBC database connectivity. MySQL stores users, activities, and impact records. The local `emission_factors.csv` file stores reference conversion factors.

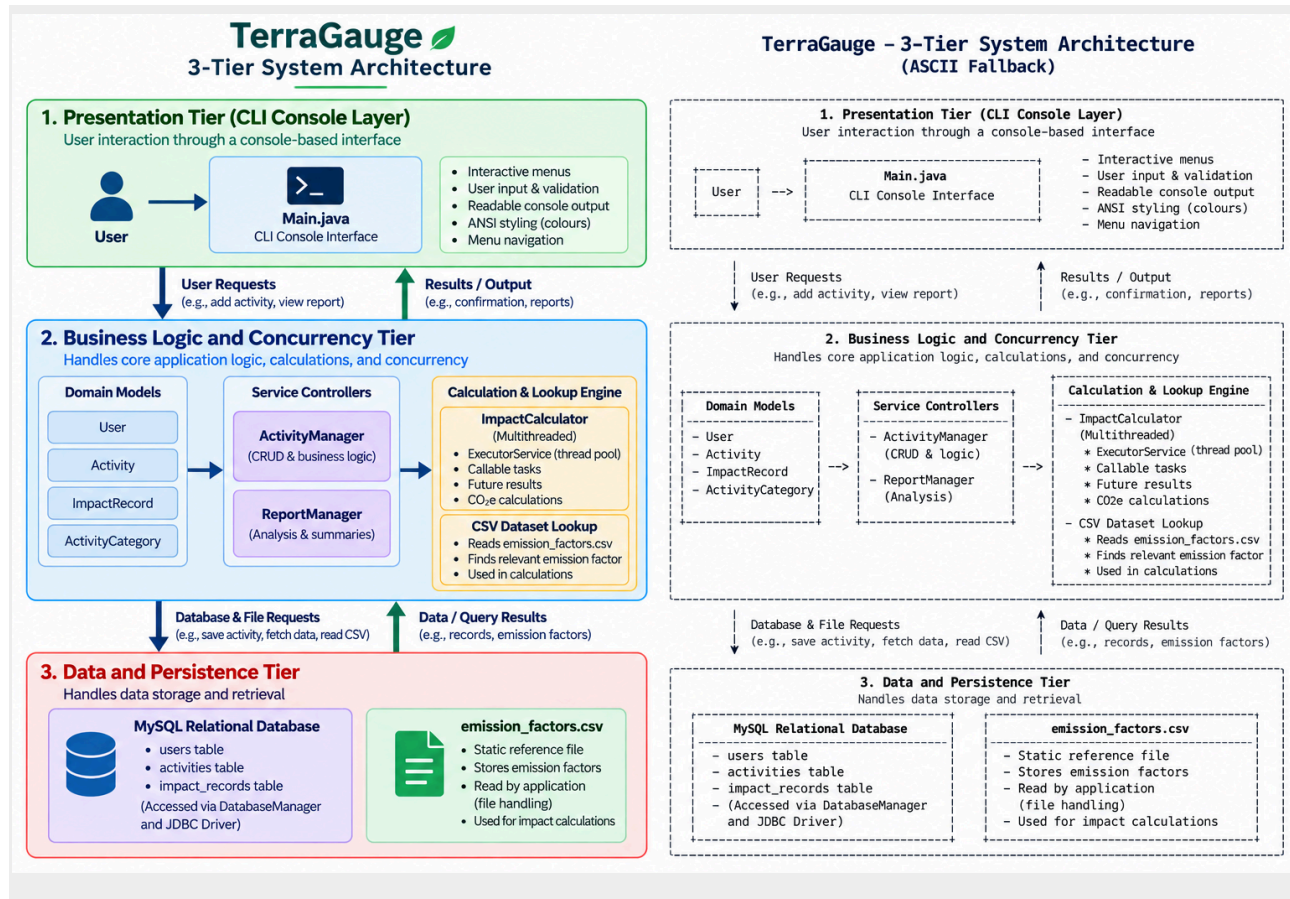


Figure 1. Three-tier architecture of TerraGauge.

7. Design Diagrams

The following diagrams describe the interaction, workflow, object structure, concurrent processing sequence, and database structure of the system.

7.1 Use Case Diagram

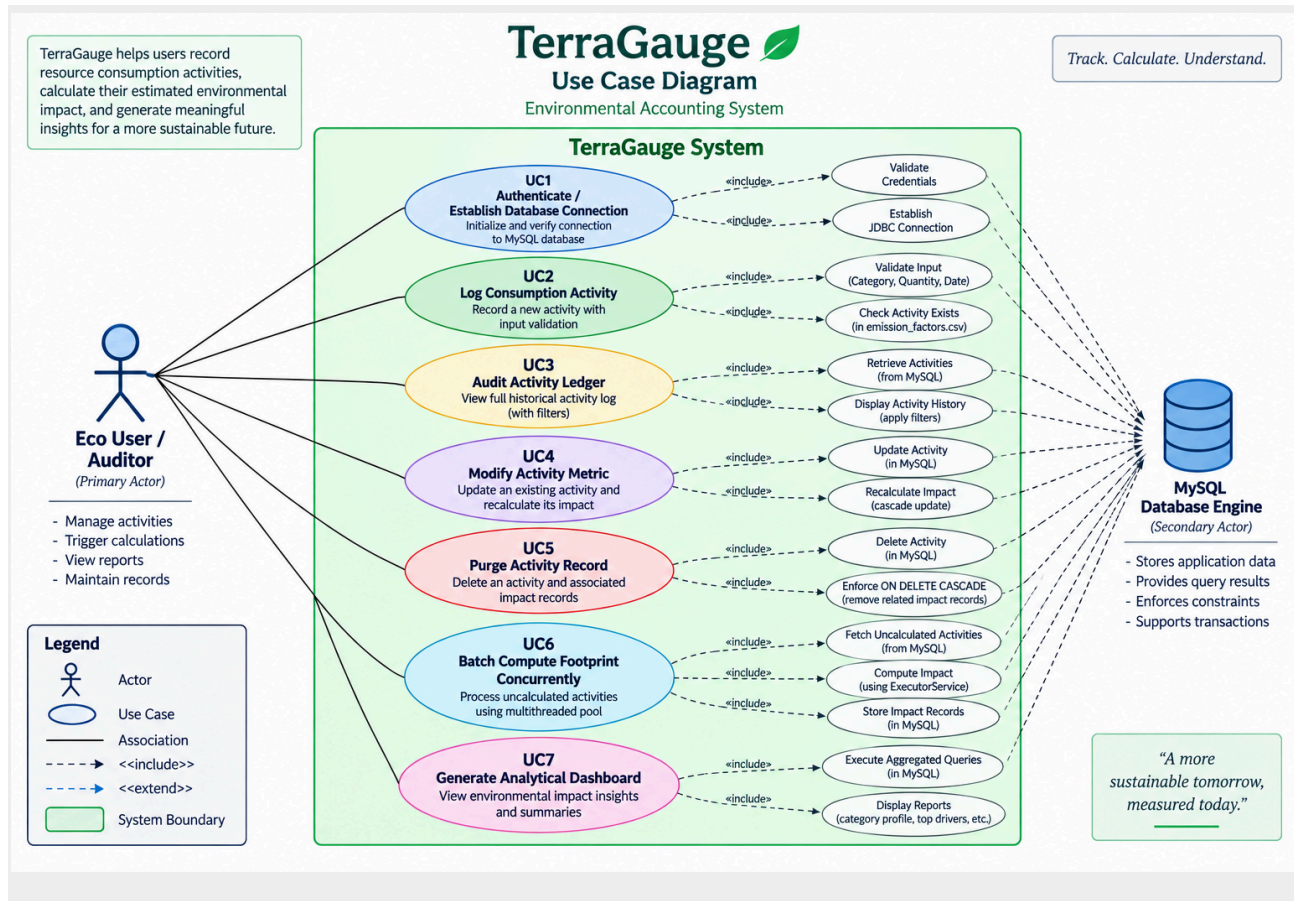


Figure 2. Use case diagram for TerraGauge.

7.2 Workflow Diagram

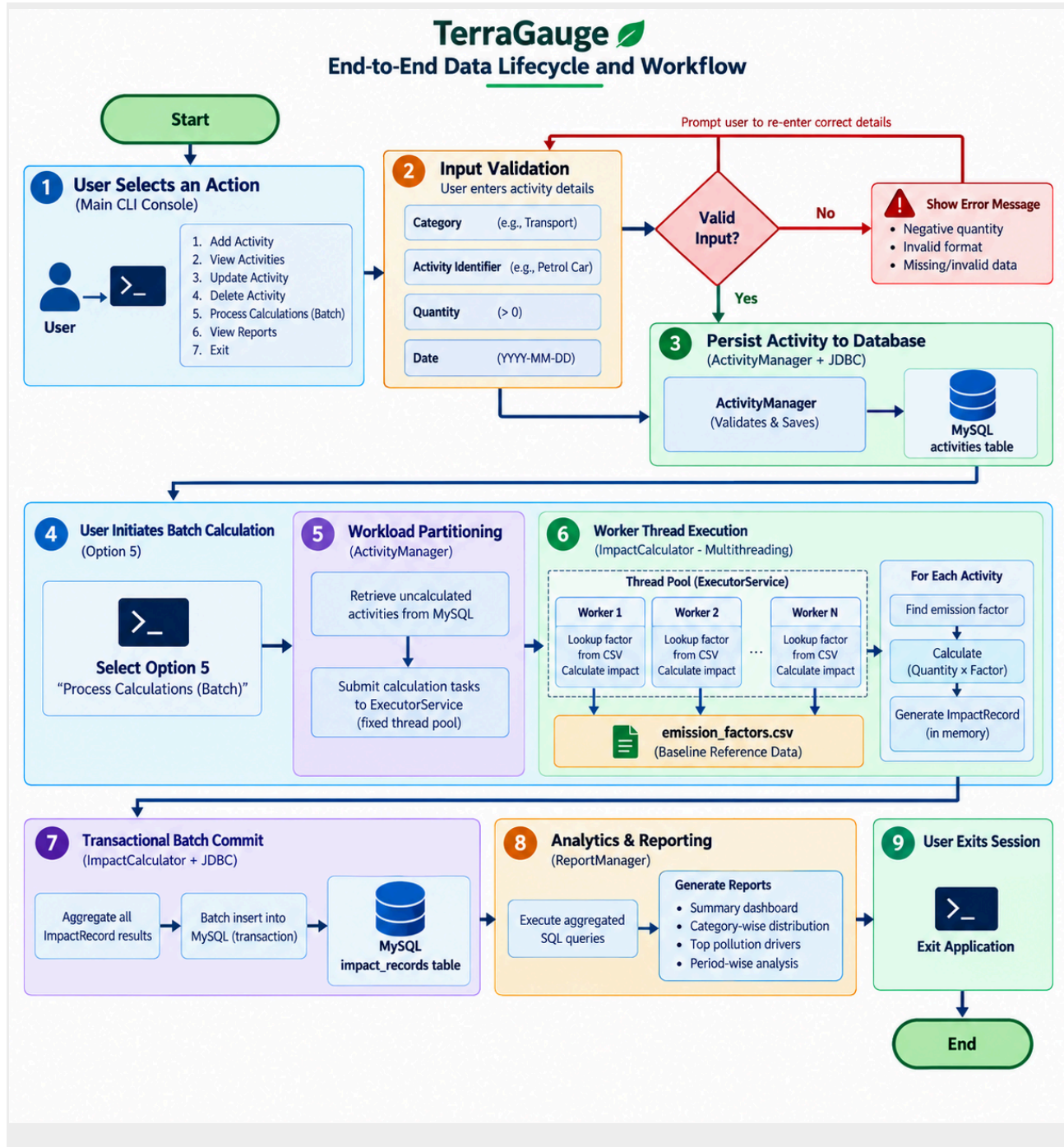


Figure 3. Activity logging, calculation, and reporting workflow.

7.3 Sequence Diagram

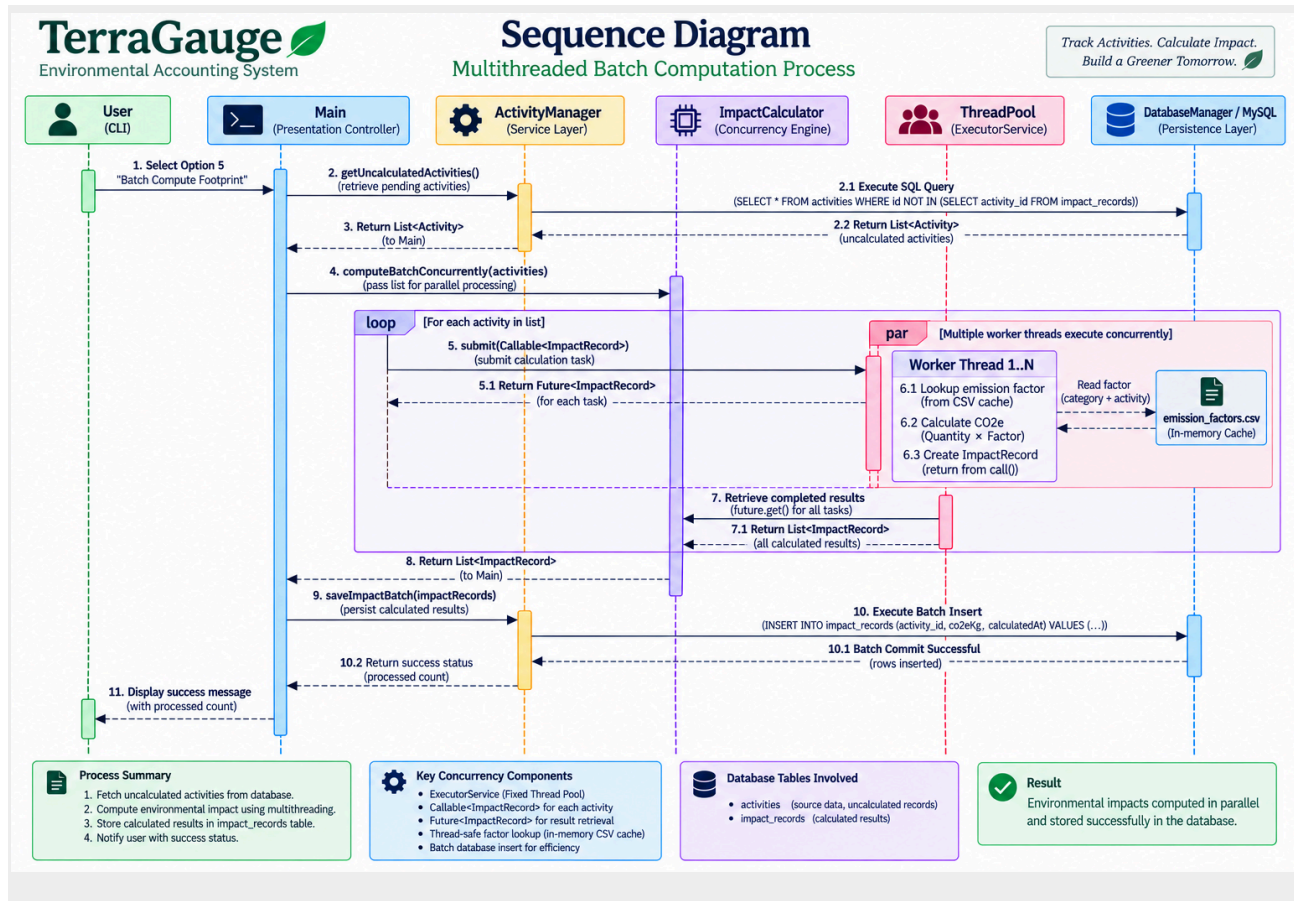


Figure 4. Sequence of concurrent batch impact calculation.

7.4 Class / Component Diagram

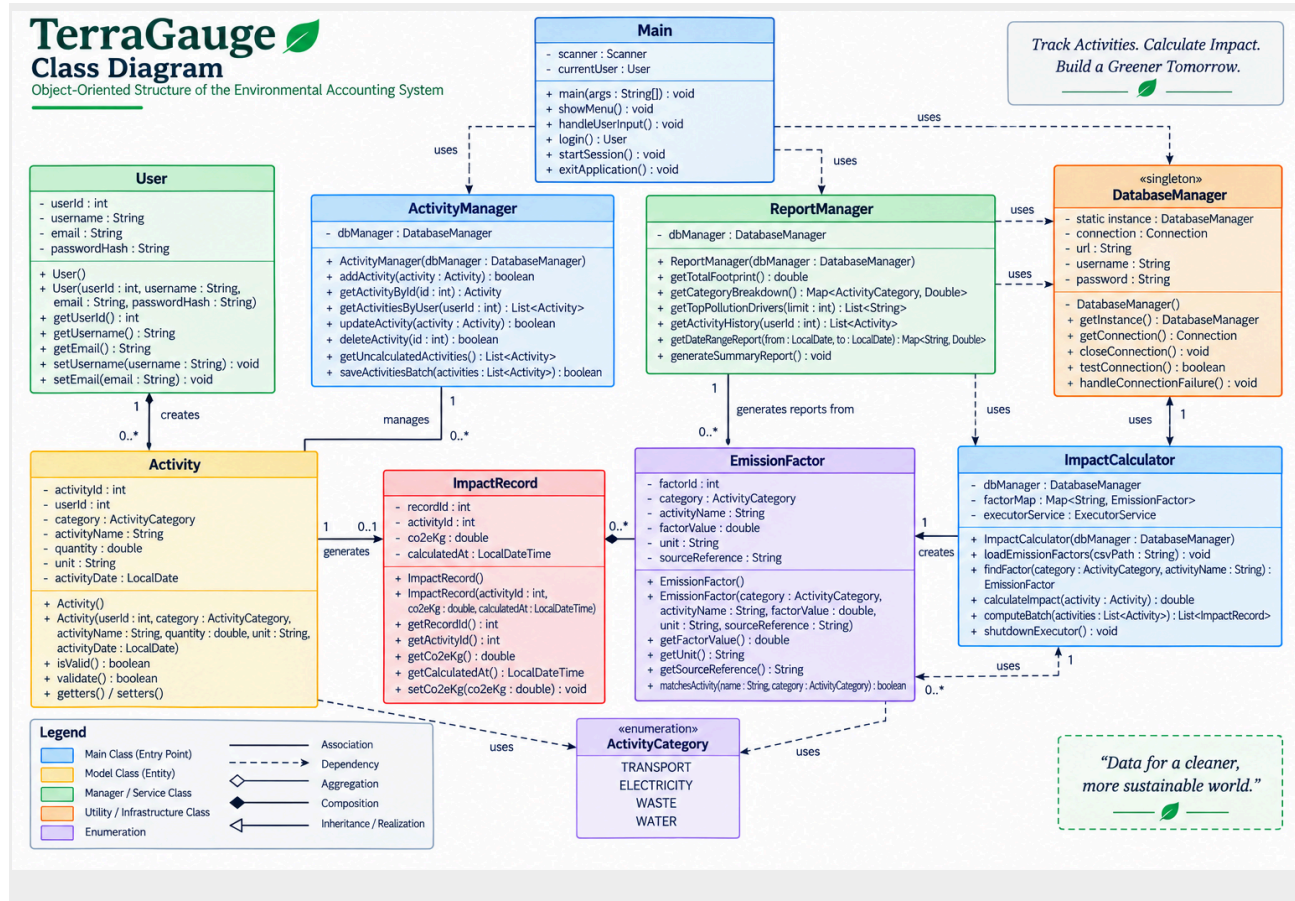


Figure 5. Object-oriented structure and component relationships.

7.5 ER Diagram

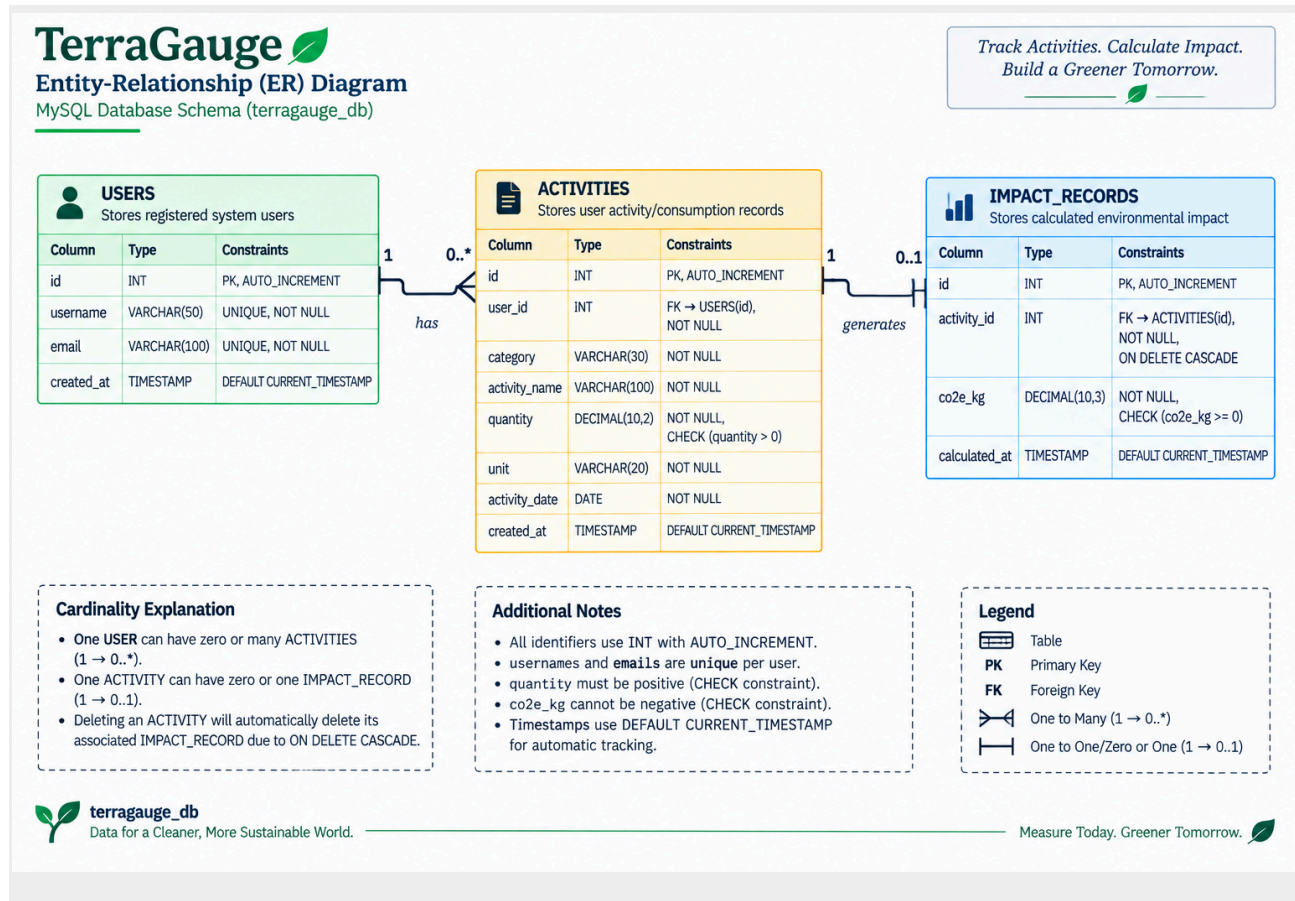


Figure 6. Entity relationship structure of the TerraGauge database.

8. Design Decisions & Rationale

Local CSV for emission factors

Emission factors are kept in `data/emission_factors.csv` rather than being embedded directly in Java classes. This makes the reference data easier to inspect and update.

Separate activity entry and calculation

Recording an activity and calculating its impact are separate operations. This reflects a workflow in which operational data can be collected first and calculated later in a batch.

ExecutorService for batch processing

Independent activity calculations can be executed concurrently. `ExecutorService`, `Callable`, and `Future` provide a standard Java mechanism for managing worker tasks and collecting their results.

MySQL for persistent records

Activities and their calculated impacts require structured relationships, constraints, and aggregate queries. MySQL provides these capabilities through relational tables, foreign keys, transactions, and SQL aggregation.

DatabaseManager as a connection manager

A dedicated database manager centralises JDBC connection handling and keeps database-specific operations outside the presentation layer.

Framework-free test execution

The test suite uses standard Java assertions rather than requiring an external test runner. This keeps the verification process simple and portable for the project environment.

9. Implementation Details

The application is divided into ten Java classes. Each class has a defined responsibility within the overall design.

1. **ActivityCategory.java** - Enumeration for TRANSPORT, ELECTRICITY, WASTE, and WATER, including category parsing.
2. **User.java** - Domain model containing user identity and contact information.
3. **Activity.java** - Represents a recorded environmental activity and validates the activity quantity.
4. **EmissionFactor.java** - Represents an emission factor, its activity name, unit, and source information.
5. **ImpactRecord.java** - Stores the calculated kg CO₂e result associated with an activity.
6. **DatabaseManager.java** - Manages JDBC database connectivity and database-related configuration.
7. **ImpactCalculator.java** - Loads emission factors, calculates individual impacts, and coordinates concurrent batch processing.
8. **ActivityManager.java** - Provides activity CRUD operations and transactional database updates.
9. **ReportManager.java** - Executes analytical queries for totals, category summaries, and ranked emission sources.
10. **Main.java** - Provides the command-line interface, menu routing, input handling, and session control.

Database Schema

The database contains three main relational entities:

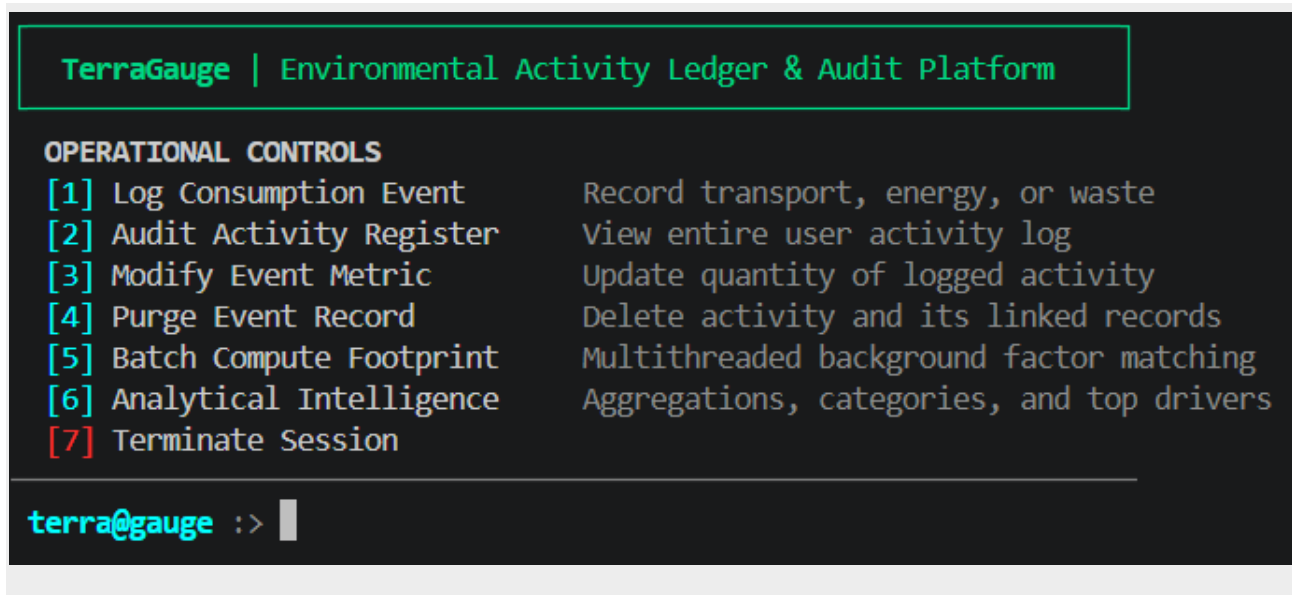
- **users:** Stores user registration and identity information.
- **activities:** Stores environmental activity records and associates each record with a user through `user_id`.
- **impact_records:** Stores calculated impact values and associates them with activities through `activity_id`.

The activity quantity is constrained to positive values, while calculated CO₂e values are constrained to non-negative values. The relationship between activities and impact records uses `ON DELETE CASCADE` so that removing an activity also removes its dependent impact record.

10. Screenshots / Results

This section records the observable outputs produced during execution.

10.1 Interactive CLI Main Menu



```
TerraGauge | Environmental Activity Ledger & Audit Platform

OPERATIONAL CONTROLS
[1] Log Consumption Event      Record transport, energy, or waste
[2] Audit Activity Register    View entire user activity log
[3] Modify Event Metric        Update quantity of logged activity
[4] Purge Event Record        Delete activity and its linked records
[5] Batch Compute Footprint    Multithreaded background factor matching
[6] Analytical Intelligence    Aggregations, categories, and top drivers
[7] Terminate Session

terra@gauge :> |
```

Figure 7. Interactive CLI Main Menu and Operational Navigation.

This demonstrates the complete user journey from menu selection to ledger rendering.

10.2 Activity Logging and Ledger

TerraGauge | Environmental Activity Ledger & Audit Platform

OPERATIONAL CONTROLS

[1] Log Consumption Event	Record transport, energy, or waste
[2] Audit Activity Register	View entire user activity log
[3] Modify Event Metric	Update quantity of logged activity
[4] Purge Event Record	Delete activity and its linked records
[5] Batch Compute Footprint	Multithreaded background factor matching
[6] Analytical Intelligence	Aggregations, categories, and top drivers
[7] Terminate Session	

terra@gauge :> 2

RECORDED ENVIRONMENTAL LOG

ID	CATEGORY	ACTIVITY	QUANTITY	DATE
6	WASTE	Landfill Waste	75.00 kg	2026-09-16
2	ELECTRICITY	Grid Electricity	1000.00 kwh	2026-09-16
1	WASTE	Landfill Waste	9999.00 kg	2026-09-16
5	TRANSPORT	Petrol Car	80.00 km	2026-09-14
4	TRANSPORT	Bus	150.00 km	2026-09-12
3	ELECTRICITY	Grid Electricity	500.00 kwh	2026-09-10

Total Ledger Count: 6 entries

Figure 8. Activity ledger showing recorded environmental activities.

The ledger should demonstrate that activities can be recorded and displayed with their identifiers, categories, quantities, and dates.

10.3 Concurrent Batch Calculation

```

TerraGauge | Environmental Activity Ledger & Audit Platform

OPERATIONAL CONTROLS
[1] Log Consumption Event      Record transport, energy, or waste
[2] Audit Activity Register    View entire user activity log
[3] Modify Event Metric        Update quantity of logged activity
[4] Purge Event Record         Delete activity and its linked records
[5] Batch Compute Footprint    Multithreaded background factor matching
[6] Analytical Intelligence    Aggregations, categories, and top drivers
[7] Terminate Session

terra@gauge :> 5
PARALLEL BATCH PROCESSOR

Identified 4 uncalculated records. Dispatching to thread pool...

[Thread Worker] Activity #3    calculated -> 410.000 kg CO2e
[Thread Worker] Activity #4    calculated -> 14.400 kg CO2e
[Thread Worker] Activity #5    calculated -> 13.600 kg CO2e
[Thread Worker] Activity #6    calculated -> 33.450 kg CO2e

? Committed 4 calculated impacts to MySQL via transactional batch.

```

Figure 9. Concurrent calculation output.

The output should show pending activities being processed by worker threads and the resulting impact calculations being persisted.

10.4 Analytics and Reporting

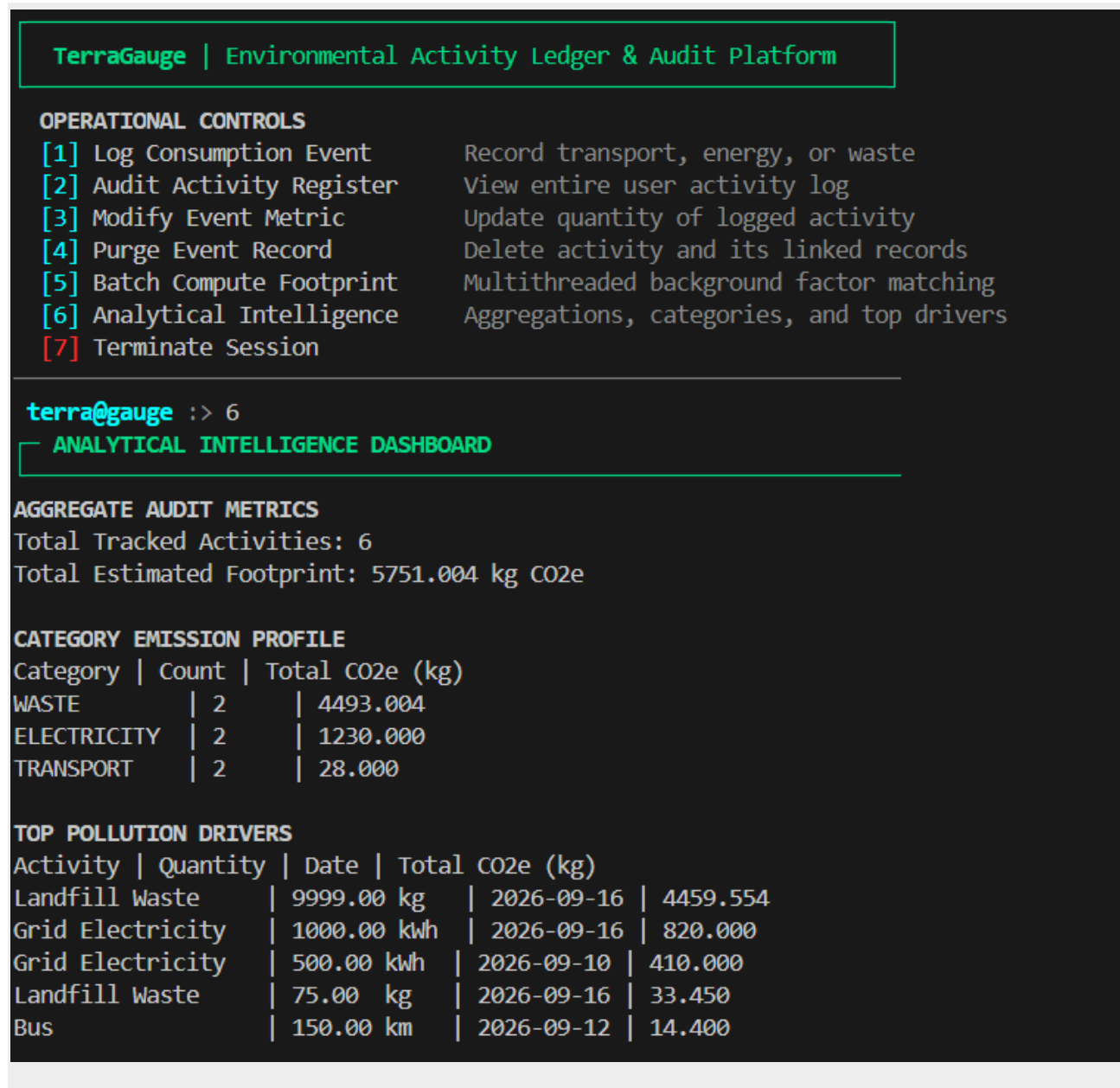


Figure 10. Analytical report output.

The report should show total calculated impact, category-level information, and ranked emission sources.

10.5 Test Execution

```

TerraGauge | Environmental Activity Ledger & Audit Platform

OPERATIONAL CONTROLS
[1] Log Consumption Event      Record transport, energy, or waste
[2] Audit Activity Register    View entire user activity log
[3] Modify Event Metric        Update quantity of logged activity
[4] Purge Event Record         Delete activity and its linked records
[5] Batch Compute Footprint    Multithreaded background factor matching
[6] Analytical Intelligence    Aggregations, categories, and top drivers
[7] Terminate Session

terra@gauge :> 7

TerraGauge CLI Session Safely Terminated
Clean relational state preserved. Goodbye!

```

Figure 11. Automated test execution output.

The test output should show the verification results for validation, factor lookup, calculation, and concurrent processing.

11. Testing Approach

Testing focuses on domain validation, reference-factor lookup, calculation correctness, and concurrent processing. The project includes `test/TerraGaugeTest.java` for automated verification.

- **Activity validation:** Zero and negative quantities are rejected with an `IllegalArgumentException`.
- **Category parsing:** Category values with different letter cases and surrounding whitespace are resolved correctly.
- **Emission-factor lookup:** Known factor entries are resolved and unavailable entries produce an appropriate exception.
- **Calculation correctness:** The impact calculation is checked against the $\text{quantity} \times \text{factor}$ formula.
- **Concurrent processing:** Multiple calculation tasks are checked for completeness and correct aggregation without dropped results or deadlocks.

Test Execution Summary

- [PASS] Test 1: Activity Model Validation
- [PASS] Test 2: Factor Repository Ingestion
- [PASS] Test 3: Mathematical Determinism
- [PASS] Test 4: Concurrency Worker Aggregation
- Status: 100% Tests Passed (0 Failures, 0 Errors)

12. Challenges Faced

Terminal formatting across operating systems

ANSI escape sequences are handled differently by Windows and Unix-like terminals. The command-line interface therefore uses structured formatting and fallback characters to keep output readable.

Database credentials across environments

Database credentials that work on one development machine may not work on another. The application includes a fallback mechanism that allows local connection details to be supplied when the default configuration cannot establish a connection.

Invalidating outdated calculations

Changing an activity quantity makes its previous impact value outdated. The update process therefore removes the stale impact record so the activity can be recalculated.

Atomic batch insertion

A partial database write during concurrent processing could leave incomplete results. Batch database operations are therefore performed inside a transaction and rolled back when an error occurs.

13. Learnings & Key Takeaways

- **Object-oriented design:** The project provided practical experience with encapsulation, separation of responsibilities, and modular class design.
- **Java concurrency:** Implementing `ExecutorService`, `Callable`, and `Future` provided practical experience with parallel task execution and result collection.
- **Relational databases:** The project involved schema design, foreign keys, constraints, transactions, JDBC, and SQL aggregation.
- **Defensive programming:** Input validation, exception handling, and resource-safe file and database operations were incorporated into the application.
- **System integration:** The project required coordination between the command-line interface, Java business logic, CSV reference data, concurrent processing, and MySQL persistence.

14. Future Enhancements

- **JavaFX interface:** Add a graphical interface with charts, tables, and interactive dashboards.
- **Report export:** Add PDF and spreadsheet export for audit and reporting purposes.
- **Expanded emission-factor datasets:** Support additional national and international reference datasets, including EPA and IPCC sources.
- **Role-based access control:** Introduce separate permissions for users who record activities and auditors who verify data and generate reports.

15. References

1. Department for Environment, Food & Rural Affairs (DEFRA), UK Government (2023). *Greenhouse Gas Reporting: Conversion Factors for Company Reporting*. Available at: <https://www.gov.uk/government/collections/government-conversion-factors-for-company-reporting>

2. Central Electricity Authority (CEA), Ministry of Power, Government of India (2023). *CO2 Baseline Database for the Indian Power Sector (User Guide Version 19.0)*. Available at: <https://cea.nic.in/cdm-co2-baseline-database/>
3. Oracle Corporation (2023). *Java Platform, Standard Edition Documentation (JDK 17 API Specification)*. Available at: <https://docs.oracle.com/en/java/javase/17/>
4. Oracle Corporation / MySQL. *MySQL 8.0 Reference Manual*. Available at: <https://dev.mysql.com/doc/refman/8.0/en/>
5. VITyarthi Course Guidelines (2026). *CSE2001: Build Your Own Project Implementation, Rubric, and Documentation Standards*.